

Context as a Cellular Automaton: A Survey of Local-Rule Memory and Context Management for LLM Agents

Gong ZhengQin
zhengqingong@gmail.com

October 2026

Abstract

Long-horizon LLM agents must decide, at every step, which fragments of an unbounded past enter a bounded context window, and which fragments of a new interaction are worth writing back. The literature answers these questions with three families of memory systems: atomic-fact stores retrieved by embedding similarity, explicit knowledge graphs traversed by diffusion or by an LLM, and entity-centric narrative profiles. We argue that, beneath their surface differences, most of these systems implement the same object: a discrete-time, local-rule dynamical process on a graph of memory units, in which each unit holds a small state, observes its neighbours, applies a shared transition rule for a bounded number of steps, and a budgeted readout selects what the language model finally sees. This is the definition of a (graph) cellular automaton. We make the reading explicit with a seven-component formalism (cells, neighbourhood, state, transition rule, schedule, readout, training signal) and use it to re-describe representative systems from atomic, graph and profile memory, from multi-hop retrieval, and from KV-cache eviction. The formalism exposes that the transition rule is almost always either a hand-set threshold on cosine similarity or a full LLM call, with nothing in between; that cell state is usually binary (in or out of context) rather than continuous; that fan-out is rarely controlled; and that per-hop evidence annotations, which several benchmarks already provide, are trajectory supervision that no memory system currently trains its rule on. We survey candidate transition functions along a cost axis, from linear diffusion through learned message passing to LLM calls, and argue that recently released small non-generative decision models, which return calibrated probabilities for typed questions in a single encoder pass, occupy the missing middle. We connect the resulting picture to neural cellular automata, to spreading-activation theories of human memory, and to latent multi-agent communication. A pilot on 300 multi-hop questions with per-hop labels tests the central falsifier, a same-budget one-shot reranker, against the iterated threshold rule of a current profile-memory system: the released rule percolates to 96% of its store and a $9\times$ readout, and at matched budget a 22M-parameter cross-encoder applied once selects better cells than the iterated rule (+0.15 chain recall, 95% CI [+0.11, +0.19]) with no measurable difference in answer accuracy; the result survives removing the narrative layer, going to atomic cells, and adding distractors. The neighbourhood earns its place only when the rule reads the *edge*: the same encoder gating on the seed’s sentence about the candidate, concatenated with the candidate, beats the one-shot ranker by +0.16 chain recall at forty cells ($K \geq 2$: +0.22 [+0.15, +0.29]), an input a global ranker cannot afford to construct. The write path tells a complementary story: when a stored relationship changes, the released rule updates only the cells named in the session and leaves 61% of the cells holding the old fact untouched, whereas propagation along the neighbourhood restores consistency above 95%, at a selectivity cost that only a trained gate can pay. Locality is a cost where a global reader exists and the rule ignores the edge; it is an advantage where the rule reads the edge, and the only means of reach where no global reader exists. We close with an experimental agenda for the transition rule that these results motivate.

1 Introduction

An LLM agent that persists across sessions faces two recurring decisions. On the read path, given a query, it must choose a subset of its accumulated memory that fits in the context window and contains what the answer needs. On the write path, given a new interaction, it must decide what to store, in what form, and what existing memory to revise or discard. Both decisions are made under a budget: tokens on the read side, LLM calls and storage on the write side.

The memory literature of 2024–2026 offers three broad answers. *Atomic-fact* systems decompose dialogue into short propositions and retrieve them by embedding similarity [8,65,68]. *Graph* systems extract entities and relations at write time and traverse the resulting structure at read time, by personalised PageRank [45], by community hierarchy [39,59], or by an LLM that walks the graph [88]. *Profile* systems aggregate facts into entity-centric narratives and retrieve the narrative [10,67,80]. Benchmarks have followed the same split: LoCoMo and LongMemEval measure precise recall over long conversations [61,62]; MemHop measures multi-hop association across entities [80]; GraphRAG-Bench asks when a graph helps at all [41].

These families are usually presented as competing architectures. This survey argues that they are better understood as instances of one object. Consider the read path of a profile system such as ProGraph [80]. Entity profiles are scored by cosine similarity to the query; the top few are selected; each selected profile is scanned for the names of other entities; a named neighbour joins the selection if its own similarity to the query exceeds a threshold; the scan repeats for a bounded number of rounds; the selected profiles and their attached facts are concatenated into the prompt. Written out, this is a threshold automaton on the graph whose vertices are entities and whose edges are co-mentions: a binary cell state (selected or not), a neighbourhood defined by the text of the profile, a local rule (become active if a neighbour is active and my relevance exceeds a constant), synchronous updates, a step budget, and a readout. The same reading applies, with different rules, to personalised PageRank over an extracted knowledge graph [45], to iterative retrieve-then-reason loops [50,83], to heavy-hitter eviction inside the KV cache [44,85], and even to the recency–importance–relevance scoring of generative agents [34], which is the degenerate zero-step case.

We call the general object a *Context-CA*, and we claim three things for the reading.

First, it is unifying without being vacuous. Once the seven components are named, systems that look unrelated become comparable on each component, and design choices that were implicit become explicit: whether the state is binary or continuous, whether the neighbourhood is fixed at write time or discovered at read time, whether fan-out is controlled, and what schedule terminates the process.

Second, it exposes a gap in the transition rule. In current systems the rule is either a hand-set threshold on an embedding score, which is cheap but blind, or a call to the language model itself, which is expressive but costs a generation per cell per step. Neural cellular automata [37,76] show that local rules can be learned; cognitive models of spreading activation [9,21] show that they can be continuous and decaying; and a recent class of small non-generative decision models, which answer typed questions over a text state with calibrated probabilities in one encoder pass [56,73], provides a rule that is learnable, batched, and orders of magnitude cheaper than a generation. Small trained gates have begun to appear at the admission step of memory systems [58,66], but no system we know of applies one as the per-cell, per-step rule of an iterated process.

Third, it turns existing annotations into training signal. Multi-hop benchmarks with per-hop gold evidence [49,75,80] tell us which memory unit should be active at which step. Under the Context-CA reading this is trajectory supervision for the transition rule, and because the rule lives in a small model while the language model is a frozen readout, the rule can be trained without

backpropagating through the LLM.

The survey is organised as follows. Section 2 reviews the three source literatures: cellular automata and their neural and graph variants; spreading activation and attractor memory in cognitive science; and LLM agent memory and its benchmarks. Section 3 defines the Context-CA formalism and re-describes representative systems under it. Sections 4–7 survey the design space component by component: transition functions along a cost axis; state design, including the decomposition of memory into a compressed narrative and precision-carrying residuals; schedule, budget and readout, including fan-out control and the cost of write-path rewriting; and training signals and evaluation. Section 8 reports a pilot experiment on the one public benchmark with per-hop labels, testing the iterated threshold rule of a current profile-memory system against a one-shot ranker at matched budget. Section 9 relates the picture to latent multi-agent communication, where cells become agents and the channel widens from a scalar to a hidden state. Section 10 lists open problems and a concrete experimental agenda; Section 11 concludes.

What this survey does not do. It does not propose a new system, and it does not claim that every memory system is *best* understood as an automaton. It claims that the automaton reading is available for most of them, that it is informative about cost and about missing design choices, and that it points to experiments that have not been run.

2 Background

2.1 Cellular automata, neural cellular automata, graph cellular automata

A cellular automaton is a set of cells with a finite state, a neighbourhood relation, and a transition rule applied to every cell in parallel at discrete time steps; the same rule at every cell is what distinguishes it from a general dynamical system [93]. Classical results concern the rule, not the substrate: simple local rules produce global structure, some rules are computationally universal, and the space of rules has qualitatively distinct regimes. Threshold automata, in which a cell becomes active when enough neighbours are active, are the simplest non-trivial family; their percolation behaviour on lattices and graphs is well characterised [16], and the linear threshold model of influence spread is the same object on social networks [53]. Two properties of threshold automata matter for what follows: activation is monotone (a cell never deactivates), which guarantees termination in at most $|V|$ steps, and the final active set is extremely sensitive to the threshold near the percolation point, which is the formal version of “the gate either admits nothing or everything”.

Neural cellular automata replace the hand-written rule with a small network trained end to end so that the automaton grows or repairs a target pattern [76]. Three design decisions in that work recur in this survey. The rule is *residual*: a cell’s state at the next step is its current state plus a learned increment, which keeps the dynamics smooth and lets the same rule serve as identity when nothing should change. Updates are *stochastic and asynchronous* in training, which makes the learned rule robust to scheduling. And training uses a *pool* of intermediate states so that the rule is supervised along trajectories, not only at a final step. Graph neural cellular automata generalise the substrate from a grid to an arbitrary graph, with the rule implemented as message passing over neighbours [37], and later work adds equivariance [29], private per-cell channels that are not exposed to neighbours [30], and rules that grow or prune the topology itself [24]. Attractor analysis of neural cellular automata is recent and, so far, confined to grids: trained NCAs are found to settle into oscillatory or quasi-periodic attractors rather than fixed points [77, 78], and we found no corresponding analysis for graph NCAs. This matters because a memory system that iterates a learned rule inherits whatever instability the rule has.

2.2 Spreading activation and attractor memory

Long before neural cellular automata, cognitive psychology modelled semantic memory as a graph of concepts in which retrieval is the spread of a continuous activation quantity along edges, decaying with distance and time, with the retrieved set being whatever exceeds a threshold when the spread stops [9, 21]. The ACT-R architecture made this operational [3], with base-level activation that decays with disuse and associative activation that flows from cues to related chunks. Three features of these models are worth carrying over. Activation is continuous, so partial relevance accumulates rather than being decided once. Spread is *gated by the cue*: a concept that is structurally adjacent but semantically irrelevant to the current cue receives little activation. And decay is a first-class part of the rule, which is how forgetting and interference are modelled without an explicit eviction policy.

A second cognitive lineage treats memory as an attractor network: a stored pattern is a fixed point of a recurrent update, and retrieval is convergence from a partial cue [48]. Modern Hopfield networks show that the attention operation itself is one step of such an update, which gives a formal bridge between “iterate a local rule until it settles” and “run attention over a set”. For this survey the attractor view supplies the vocabulary for the termination question: a Context-CA that runs to a fixed point is performing pattern completion over memory units, and its failure modes, spurious attractors and non-convergence, are the classical ones.

2.3 Long-term memory for LLM agents

Agent memory systems separate a *write path*, invoked after each interaction, from a *read path*, invoked per query. Cognitive-architecture treatments of language agents [17] partition memory into working, episodic, semantic and procedural stores and describe the paths as retrieval and learning actions; most concrete systems implement only an episodic-to-semantic pair. Recent surveys organise the field by memory form, function and dynamics [97], by graph representation and evolution [96], and by the broader practice of context engineering [94]; none of them treats the read and write paths as a dynamical process, which is the gap this survey addresses.

On the write path, atomic-fact systems run one or two LLM calls per interaction to extract propositions and to decide whether each proposition adds, updates or deletes an existing one [65]; Zettelkasten-style variants additionally link the new note to existing notes and may rewrite the neighbours [8]. Graph systems run an extraction call that emits entities and relations, followed by resolution and clustering [39, 45], or maintain a temporally indexed graph in which facts carry validity intervals [98]. Operating-system analogies page memory between a bounded context and external storage under explicit control of the model [95]. Profile systems run one call per mentioned entity that rewrites the entity’s narrative to incorporate the new interaction [10, 80]. Recent work attaches a second layer to the narrative: precise facts the narrative would paraphrase away, extracted in the same call [80], or a separation between an indexed short abstraction and an unindexed detailed value [67]. Lifecycle mechanisms — decay [32], admission control [7], hierarchical eviction [31], temporal resolution of relative dates [90] — are treated as add-ons to whichever base system they are attached to.

On the read path, atomic systems embed the query and take the top-k propositions; graph systems seed a walk from query entities and run diffusion or an LLM-guided traversal; profile systems take the top profiles and, in the most recent work, expand to co-mentioned entities. A small number of systems import the cognitive models of Section 2.2 directly: SYNAPSE runs spreading activation with lateral inhibition and temporal decay over a dynamic memory graph [87]; spreading activation has been used to replace LLM-guided traversal over automatically built knowledge graphs

[79]; and ACT-R’s activation and decay equations have been transplanted onto agent memory [47]. Multi-hop retrieval over static corpora developed the same loop earlier, alternating retrieval and reasoning [50, 64, 83] or walking an explicit graph [42, 88]. Inside the model, KV-cache management makes analogous decisions at token granularity: which past tokens’ keys and values survive, chosen by accumulated attention [44], by recent-window attention [85], or by position [86].

Benchmarks are dominated by precise single-hop recall over long conversations [61, 62], with recent additions for multi-hop association across entities with per-hop evidence [80], for incremental multi-turn ingestion [69], and for the prior question of whether a graph helps at all [41]. Evaluation has converged on LLM-as-judge accuracy, whose known biases toward verbose or well-formatted answers have prompted unbiased pairwise protocols [12].

3 The Context-CA formalism

3.1 Definition

A Context-CA is a tuple $(V, E, S, f, \sigma, \rho, L)$:

- **Cells V .** The units of memory over which selection is decided. Candidates in the literature are propositions, chunks, entities with profiles, community summaries, tree nodes, or individual tokens’ KV entries. The choice fixes the granularity at which the readout can include or exclude material.
- **Neighbourhood E .** A relation on V that determines which cells can influence which. It may be fixed at write time (extracted relations, Zettelkasten links, tree parenthood, token adjacency) or discovered at read time (co-mention of names inside a narrative, cosine adjacency, shared community).
- **State S .** What each cell carries between steps. At minimum a binary selection bit; more generally a continuous activation, a heat over attached sub-units, a hop index, or a small vector. The text payload of the cell is usually treated as a constant parameter of the cell rather than as state.
- **Transition rule f .** A function from a cell’s own state, its payload, its neighbours’ states and the query to the cell’s next state. The same f is applied at every cell; this is the cellular-automaton constraint and the reason the rule can be cheap and batched.
- **Schedule σ .** Synchronous or asynchronous application of f , the step budget H , and the termination test (budget exhausted, active set stable, or activation below a floor).
- **Readout ρ .** The map from the final states to the prompt: which cells’ payloads are included, in what order, and under what token budget B .
- **Training signal L .** What, if anything, f is fitted to. Most systems have none; f is hand-set.

The write path is the same object with a different driver: the incoming interaction, rather than a query, seeds activation, and the readout is the set of cells whose payload is rewritten or whose sub-units are admitted, evicted, or superseded.

Two clarifications keep the definition honest. A single embedding lookup with top-k is a Context-CA with $H = 0$: f is applied once, there is no neighbourhood, and ρ takes the top-k. Calling it an automaton adds nothing by itself; the reading becomes informative only when $H \geq 1$ or when the write path has a non-trivial rule, and the value of the formalism is that it makes the $H = 0$ case’s missing components visible as choices rather than as absences. Second, the “same rule everywhere” constraint is a constraint on f ’s *parameters*, not on its inputs: f sees the cell’s own payload, so different cells behave differently, as they do in every cellular automaton.

3.2 Worked mappings

Threshold expansion over co-mention (ProGraph). V = entities; $E = \{(i, j) : \text{the name of } j \text{ appears in the profile of } i\}$, discovered at read time by string matching; $S = \{0, 1\}$; $f(i) = 1$ if $s(i) = 1$, else 1 if some neighbour j with $s(j) = 1$ exists and $\cos(q, p_i) > \tau$; σ synchronous with $H = 5$; ρ concatenates the profiles of active cells plus, per cell, the top-8 attached facts by cosine to the query; $L = \text{none}$, τ and H hand-tuned. This is a bootstrap-percolation process on the co-mention graph with a per-cell threshold on a query-conditioned score [80]. The paper’s own sensitivity analysis reports that the active set grows from roughly 8 to roughly 20 cells as H is raised from 1 to 5 on five-hop questions, and that accuracy on those questions *falls*, which is the signature of a threshold automaton past its percolation point: the gate passes everything reachable, and the readout is left with a context that approaches the full store. The released implementation additionally applies the gate to the *maximum* over candidate neighbours and then admits all of them, so the per-cell threshold is in practice a per-step threshold; fan-out is all-or-nothing at each hop.¹

Personalised PageRank over an extracted graph (HippoRAG). V = phrase nodes of an extracted knowledge graph plus passage nodes; E = extracted relations plus synonymy edges; S = a non-negative scalar mass; f is the linear diffusion $x \leftarrow (1 - \alpha) P x + \alpha x_0$ with restart vector x_0 on nodes matched to query entities; σ runs to convergence; ρ ranks passages by accumulated mass and takes the top- k [45]. The rule is linear and query-conditioned only through the seed, so a structurally central but semantically irrelevant node attracts mass. The graph is fixed at write time; extraction errors are inherited by every query, which the ProGraph authors identify as the reason PPR barely beats chunk retrieval on noisy dialogue.

Iterative retrieve-then-reason (IRCoT, Self-Ask). V = passages; E is implicit and discovered at read time through the LLM’s intermediate reasoning, which names the next thing to retrieve; $S = \{\text{retrieved, not retrieved}\}$; f is a full LLM call that reads the current context and emits a new retrieval query; σ alternates retrieval and generation for a bounded number of rounds [50,83]. Here the transition rule is maximally expressive and maximally expensive: one generation per step, over a context that grows with each step. The multi-hop dense retriever [64] replaces the LLM with a learned query encoder conditioned on the previous hop, which is the earliest example we know of a *trained* cheap transition rule for a Context-CA over documents.

LLM-guided graph walk (Think-on-Graph). V = knowledge-graph entities; E = typed relations; S = on or off the current beam; f is an LLM call that scores candidate relations and neighbours; σ is beam search with a depth budget; ρ returns the beam paths [88]. Structurally a Context-CA with a generative f and beam width as the fan-out control; the cost per cell-step is a generation.

Zettelkasten write path (A-Mem). On the write path, V = notes; a new note is a new cell; f is an LLM call that, given the new note and its nearest existing notes, decides which links to create and may rewrite the neighbours’ summaries; σ is a single asynchronous step per write [8]. Under the formalism this is developmental: the write rule grows E . The read path is $H = 0$ over the resulting graph.

Cue-anchor traversal (Memora). V = memory entries, each with an indexed short abstraction, an unindexed detailed value, and indexed cue anchors; E is induced by shared cue anchors; the read rule is a policy-guided retriever that iteratively refines the query and expands through anchors [67]. This is a Context-CA with a two-part state and a rule that is either LLM-guided or, in an experimental variant, RL-trained; to our knowledge it is the closest existing system to a trained transition function over memory units.

¹Our inspection of `prograph_runner.py` in github.com/ShengtongZhu/ProGraph at the commit of 2026-08-06; the paper’s worked example describes per-candidate gating.

Hierarchical dual-route retrieval (Mnemis). V = nodes of a base graph and of a hierarchical abstraction graph; two f 's run in parallel, one similarity-based on the base graph and one top-down selection on the hierarchy [72]. The design is a two-automaton system whose readouts are merged; the hierarchy is a way of coarse-graining V so that a global rule becomes affordable.

Community hierarchy (GraphRAG), tree summaries (RAPTOR). V is coarse-grained at write time into community summaries or tree nodes; the read path chooses a level and takes the nodes at that level [39, 81]. Under the formalism the coarse-graining is a choice of V , and the level selection is an $H = 0$ readout with a budget; dynamic community selection [40] adds an LLM-rated descent through the hierarchy, which is an $H \geq 1$ automaton on the tree with a generative rule.

KV-cache eviction (H2O, SnapKV, StreamingLLM). V = token positions in the cache; E = attention adjacency; S = keep or evict; f keeps a token if its accumulated attention mass is among the heaviest [44], if its attention from a recent observation window is high [85], or if it is a sink or recent [86]; σ runs once per decoding step [44, 85, 86]. These are Context-CAs on the token grid with a rule read directly off the model's own attention, which is the one setting where the rule is both cheap and informed by the model.

Recency–importance–relevance scoring (Generative Agents). V = memory records; no E ; S = a scalar score; f is a weighted sum of recency decay, an LLM-rated importance stored at write time, and cosine relevance; $H = 0$ [34]. The interest of the mapping is that this is spreading activation with the spread removed: a base-level activation (recency), a stored strength (importance) and a cue match (relevance), summed, with no propagation along edges.

3.3 What the mappings show

Table 1 arranges the systems above by component. Three regularities stand out.

The rule is bimodal in cost. Column f contains either a threshold or linear operation on precomputed scores (ProGraph, HippoRAG, SYNAPSE, KV eviction, Generative Agents) or a generation by the language model (IRCoT, ToG, A-Mem's write path, dynamic community selection). The middle of the cost axis, a small trained model that reads the cell's payload and the query and returns a number, is occupied only at the edges of the process: by the multi-hop dense retriever's hop-conditioned query encoder [64], by Memora's retrieval policy [67], and by admission gates that sit between a vector store and the LLM and decide once per candidate [58, 66]. None is applied per cell per step inside an iterated expansion.

State is almost always binary. PPR and Generative Agents carry a continuous per-cell quantity but do not iterate a query-conditioned nonlinear update on it. SYNAPSE is the exception that proves the pattern: it carries continuous activation with decay and inhibition, but its rule is the hand-set spreading-activation equation, not a learned or query-reading function [87]. The spreading-activation lineage's central feature, continuous activation that accumulates and decays, has therefore entered LLM memory systems only in its 1975 form.

Fan-out is uncontrolled or controlled by generation. Where fan-out control exists, it is a beam width enforced by an LLM's ranking (ToG), a global top-k after convergence (PPR), or, in one system, lateral inhibition in the cognitive-model sense [87]. Threshold-expansion systems have no fan-out control at all, and their own sensitivity analyses show the consequence.

Training signal is absent. Column L is empty for every system except Memora's policy, the multi-hop dense retriever, and the admission gates. This is despite the availability of per-hop supporting-fact annotations in HotpotQA, MuSiQue and MemHop [49, 75, 80], which under the formalism label which cell should be active at which step.

System	Cells V	Neighbourhood E	State S	Rule f	Schedule σ	Readout ρ	Signal L
Top-k embedding (Mem0 read)	propositions	none	$\{0,1\}$	cosine top-k	H=0	top-k	none
Generative Agents	records	none	scalar	recency+ importance+ relevance	H=0	top-k	none
ProGraph read	entity profiles	co-mention (read-time, string)	$\{0,1\}$	active neighbour $\wedge$ $\cos > \tau$	sync, H=5	active profiles + top-8 facts	none
HippoRAG	phrases+ passages	extracted relations + synonymy	mass ≥ 0	linear PPR diffusion	to convergence	top-k passages by mass	none
SYNAPSE	episodic/ semantic nodes	dynamic memory graph	continuous activation	spreading activation + lateral inhibition + decay (hand-set)	iterative	above- threshold nodes	none
MemGate (admission)	candidates from vector store	none	admit/ reject	9M-param query- conditioned gate	H=0	admitted set	trained
MDR	passages	implicit	$\{0,1\}$	trained hop- conditioned encoder	H=2	top passages	supporting facts
IRCoT / Self-Ask	passages	implicit via reasoning	$\{0,1\}$	LLM generation	alternate, bounded	growing context	none
Think-on- Graph	KG entities	typed relations	on/ off beam	LLM scoring	beam, depth budget	beam paths	none
A-Mem write	notes	Zettelkasten links (grown)	note text	LLM link/ rewrite	async, 1 step	rewritten neighbours	none
Memora read	entries (abstraction+ value+ anchors)	shared anchors	selected set	LLM-guided or RL-trained retriever	iterative	selected values	RL (experimental variant)
Mnemis	base + hierarchy nodes	base edges; hierarchy	two selected sets	similarity top-down selection	parallel routes	merged	none
GraphRAG dynamic selection	community summaries	tree	on/ off	LLM rating	descent	rated communities	none
H2O / SnapKV	KV entries	attention	keep/ evict	attention- mass rule	per decode step	kept entries	none

Table 1: Representative systems as Context-CAs.

4 Transition functions along the cost axis

The transition rule is where the systems in Table 1 differ most and where the cost of the whole process is decided, because f is evaluated once per cell per step. We order the candidates by cost per evaluation and note, for each, what it can see and what it can learn.

4.1 Thresholds on precomputed scores

The cheapest rule compares a precomputed cosine similarity to a constant. It sees the query and the cell’s embedding, and nothing about the neighbours beyond their binary state. It cannot learn. Its failure mode is structural: the score distribution of a fixed embedder is narrow, a single τ must serve every query, and the process is a bootstrap percolation whose outcome flips between “nothing” and “everything reachable” near the percolation threshold [16]. ProGraph’s Appendix D documents exactly this on five-hop questions, where the active set more than doubles as the hop budget grows and accuracy falls [80]. A related hidden cost is that the threshold is tuned to one embedder’s cosine geometry; changing the embedder changes the fan-out behaviour, which the authors acknowledge.

4.2 Linear diffusion

Personalised PageRank [14, 51] and label propagation replace the threshold with a linear operator: mass flows along edges, restarts at the seeds, and the fixed point is a global ranking. The rule sees the whole graph through repeated multiplication but sees the query only through the seed vector. Cost is a sparse matrix–vector product per step, negligible next to any model call, and convergence is guaranteed. HippoRAG uses PPR over an extracted graph [45, 46]; AtomicRAG uses it over an atom–entity graph [11]; KET-RAG uses PageRank at index time to select the chunks worth extracting a graph from [54]. Diffusion’s weakness is the mirror of its strength: it is query-blind after seeding, so structurally central nodes attract mass regardless of relevance, and it inherits every write-time extraction error as a permanent edge. Spreading activation in the cognitive sense [9, 21] is the nonlinear, decaying, cue-gated relative of diffusion; SYNAPSE and the KG-RAG spreading-activation retriever are its direct implementations [79, 87], and their rules remain hand-set.

4.3 Learned message passing

Graph neural cellular automata implement f as a message-passing network trained so that iterating it produces a target [29, 37], and the message-passing framework [35] gives the general template: a message function over (own state, neighbour state, edge), an aggregation, and an update. In retrieval, GNN-RAG trains a GNN over the knowledge graph to score answer candidates and verbalises the retrieved paths for the LLM [38]; G-Retriever casts subgraph selection as a prize-collecting Steiner tree with learned node prizes [42]. These are trained, cheap per evaluation, and see the graph, but they see the query only as a vector and the cell only as an embedding; they cannot read the cell’s text. Their training targets are answer nodes, not trajectories.

4.4 The language model as the rule

At the expensive end, f is a generation. Iterative retrieve-then-reason methods have the LLM write the next query [50, 83]; Think-on-Graph has it score relations and prune beams [88]; A-Mem has it decide links and rewrite neighbours on the write path [8]; dynamic community selection has it rate summaries as it descends a hierarchy [40]; Memora’s policy-guided retriever refines the query and

Rule f	Sees query	Sees cell text	Sees neighbours	Learnable	Cost per cell-step	Examples
Cosine threshold	as vector	no	binary state	no	~0	ProGraph Stage 2
Linear diffusion	via seed only	no	whole graph	no	sparse mat-vec	HippoRAG, AtomicRAG
Spreading activation (hand-set)	via seed + decay	no	weighted	no	~0	SYNAPSE, Pavlović et al.
Hop-conditioned encoder	yes	as embedding	previous hop	yes	one encoder pass	MDR
GNN message passing	as vector	as embedding	yes	yes	one GNN pass	GNN-RAG, G-Retriever, GNCA
Admission gate	yes	as embedding	no	yes	~ms	MemGate
Typed decision model	yes	yes ($\leq 1k$ tok)	scalar / one line	yes	~10–40 ms (vendor)	Laya-class
Cross-encoder / NLI verifier	yes	yes	no	yes	~10–100 ms	monoBERT, MiniCheck
LLM generation	yes	yes	yes (as text)	via prompt	one generation	IRCoT, ToG, Memora policy

Table 2: Transition-function candidates.

expands through cue anchors [67]. This rule sees everything and can, in principle, reason about why a neighbour matters. Its cost is a generation per cell per step over a context that typically grows with the step, and its output must be parsed. Empirically, AgentPrune finds that 28–73% of inter-agent messages in LLM multi-agent pipelines can be pruned without loss [4], and the profile-memory results suggest that on noisy dialogue an LLM-scale graph walk buys little over what a narrative already encodes [80].

4.5 Small non-generative decision models

Between the threshold and the generation lies a class of models that read text and return a decision without generating: cross-encoder rerankers [19, 74], factual-consistency verifiers [6, 70], and, most recently, typed decision models that answer a set of structured questions (multiple choice, ordinal score, or the probability that a statement is true) over a text state in a single bidirectional-encoder pass [52, 56]. The latter are built on long-context encoders [71, 73], are trained with reinforcement learning against strictly proper scoring rules so that the reported probability is, in expectation, an honest belief [36], and ship with calibration that must be refitted on the target distribution [43]. Vendor-reported latency is tens of milliseconds per question on a commodity GPU and single-digit milliseconds per question batched; vendor-reported zero-shot accuracy on their own typed-decision benchmark is *below* the per-question majority-class baseline, and the usable accuracy the vendor reports belongs to a checkpoint fine-tuned on that benchmark’s own training split, a caveat the vendor states itself [56]. No peer-reviewed description of these models exists as of this writing, and the comparison baseline they cite is a commercial API whose architecture is undisclosed [52]; we treat both as software artefacts rather than as results.

For the Context-CA these models satisfy the constraints on f exactly. The same weights serve every cell. The input is local: the query plus one cell’s payload fits a 512–1024-token window, with the neighbours’ influence entering as a scalar or a one-line message rather than as text. Evaluation is batched, so one synchronous step over the whole active frontier is one forward pass. The output is a calibrated probability, which is a continuous state, and it needs no parsing. The precedent for this pattern is old: a cascade of cheap classifiers that rejects most candidates before an expensive stage [92], and LLM cascades and routers that send a query to a small model first [33, 82]. The novelty is only that the cheap stage is now expressive enough to read a memory unit and a query together.

Two constraints follow. The cell payload must fit the window, which forces the narrative half of a profile to stay short and pushes precision into the residual layer (Section 5). And the rule is only as good as its fine-tune: a zero-shot decision model is a worse gate than a cosine threshold on the vendor’s own numbers, so this class of rule is usable only where trajectory labels exist (Section 7).

5 State design: what a cell carries

5.1 Selection bit, activation, heat

The minimal state is a bit: the cell is in the context or it is not. Most systems in Table 1 stop there. A continuous activation $a \in [0, 1]$ is the next step, and it is what spreading-activation theory [9, 21] and neural cellular automata [76] both use, for the same reasons: partial evidence accumulates across steps instead of being decided once, decay is expressible as part of the rule, and the readout can rank rather than merely include. Below activation, sub-cell state is useful when a cell has internal structure: a per-fact *heat* over the facts attached to an entity, so that the readout can include the two facts relevant to this query rather than the cell’s whole payload. ProGraph’s per-entity top-8 residual selection is a query-time heat computed by cosine; making it state that persists and decays across queries is the natural extension.

5.2 The profile–residual decomposition

A recurring finding in the 2026 memory literature is that a single representation cannot serve both association and precision. Narrative summaries carry cross-entity association in their text but paraphrase dates, quantities and names away; atomic facts preserve precision but lose the narrative that makes multi-hop traversal possible. Several systems therefore split the cell payload in two. ProGraph co-extracts, in the same write-time call, an updated narrative and a set of *compression residuals*, the facts the narrative would lose, and shows by ablation that the residual layer carries the precision categories on LoCoMo while the narrative layer carries multi-hop on MemHop, with cross-effects under three points [80]. Memora separates an indexed short abstraction from an unindexed detailed value and adds cue anchors as alternative entry points [67]. TriMem keeps three granularities at once [89]. TSM adds a temporal residual, resolving relative time expressions to absolute dates at write time [90].

Under the Context-CA reading the decomposition has a clean interpretation: the narrative is the *low-frequency* component of the cell, the part that the neighbourhood relation and the transition rule read; the residuals are the *high-frequency* component that the readout includes but the rule never needs to see. This is what makes a bounded-window rule feasible. If f reads only the narrative, the narrative can be capped at a few hundred tokens without loss of precision, because precision lives in residuals that are selected by a separate, cheaper mechanism. The decomposition also explains a cost pathology: when the narrative is *not* capped, every write rewrites the whole accumulated narrative, and cumulative write cost grows quadratically in the number of sessions (Section 6.3).

5.3 Residual updates in the NCA sense

The word “residual” is used in a second sense in this literature, and the two should be kept apart. In neural cellular automata the *update* is residual: $s_{t+1} = s_t + \Delta(s_t, \text{neighbours})$, with Δ small and learned, so that the rule is the identity when nothing should change and the dynamics are smooth [76]. Applied to a memory cell’s activation this gives $a_{t+1} = a_t + \alpha \cdot r \cdot m - \beta \cdot a_t$, where r is the rule’s estimate of the cell’s own relevance, m is the neighbours’ influence, and β is decay. The two senses combine naturally: the storage residuals decide what the cell *contains*; the update residual decides how the cell’s *activation* moves. Whether a residual update over continuous activation converges on a memory graph is an open question; the only attractor analyses of trained NCAs are on grids and find oscillation rather than fixed points [77, 78].

5.4 Decay, admission, eviction, contradiction

Lifecycle mechanisms are state transitions in the write-path automaton. Decay lowers activation or heat with disuse: Ebbinghaus-style forgetting [68], adaptive decay [32], ACT-R base-level activation [47]. Admission control decides whether a new unit enters at all, framed as a decision over utility, confidence, novelty, recency and type prior [7]. Eviction removes units under a storage budget [31]. Contradiction is the transition that most systems lack: ProGraph never evicts a residual, so a superseded fact persists next to its replacement and the readout is told that residuals override the narrative [80]; Mem0’s update-or-delete decision is an LLM call per fact [65]. A verifier-class rule (Section 4.5) that answers “does this new fact contradict that stored fact” is the cheap local transition that would close the gap, and the cost of applying it scales with the number of stored residuals per cell, which is one more reason to keep cells small. Section 8.5 measures the gap: after a relationship changes, 43% of the cells that recorded it still carry a residual asserting the old version even when their narratives are correctly rewritten, and one contradiction call per cell reduces this to 17%.

6 Schedule, budget and readout

6.1 Synchronous steps, hop budgets, termination

Every iterated system in Table 1 uses synchronous steps with a hop budget: ProGraph’s $H = 5$, ToG’s depth limit, IRCot’s round limit, PPR’s convergence tolerance. The budget is doing two jobs at once, bounding cost and bounding fan-out, and the sensitivity results show it does the second job badly: raising H from 1 to 5 helps one-hop questions most (by rescuing cells that the seed step missed) and hurts five-hop questions (by admitting tangential cells), so the optimal budget is question-dependent and no system sets it adaptively [80]. Termination by stability, stopping when the active set or the activation vector stops changing, is the standard alternative in automata and attractor networks [48] and requires either monotone activation, which guarantees termination, or a damped update, which requires the convergence analysis that does not yet exist for graph NCAs. Asynchronous schedules, in which only excited cells update, are the natural fit for the write path, where a new session touches a few entities; NCA training with stochastic asynchronous updates [76] and self-stabilisation under asynchronous local rules [26] are the relevant precedents.

6.2 Fan-out control

The mechanisms available are: a global cap (top-k after ranking), a beam (ToG), a per-hop threshold (ProGraph, which in the released code gates on the best candidate and then admits all candidates), and lateral inhibition, in which a cell’s activation suppresses its siblings’ so that only the strongest of a set of competing neighbours survives [87]. Lateral inhibition is the only one of these that is *local*, that is, expressible inside f , and therefore the only one that a cellular-automaton rule can learn. It is also the mechanism that spreading-activation theory uses to explain why dense neighbourhoods do not swamp retrieval [9].

6.3 Budgeted readout and the cost of what is not in the window

The readout turns final states into a prompt under a token budget B . Existing readouts are concatenations ordered by score, with the well-documented position effects of long prompts [63] left to the model. Prompt-compression methods that drop low-information tokens [60, 84] and agent-loop compaction methods that summarise the working context under a budget [1, 2, 20] operate on the

readout after selection; under the formalism they are a second, finer-grained automaton over tokens, and their interaction with cell-level selection is unstudied. The readout interacts with the transition rule in a way that is easy to miss: if the rule admits most of the store, the readout approaches the full-context baseline, and the whole apparatus has cost more than a single long prompt. On a corpus of a few thousand tokens per scenario, a profile system that ties the full-context baseline in accuracy can exceed it in read tokens by a wide margin: in the pilot of Section 8 the released rule’s readout averaged 9.1 times the dialogue it was built from,² while adding a write-path cost that the baseline does not have. The benefit of selection appears only when the corpus is much larger than the selected context, and the crossover has not been measured.

The write path has its own budget pathology. When the cell payload is a narrative that is rewritten in full at every mention, the tokens per rewrite grow with the number of prior sessions, and the cumulative cost over T sessions is $O(T^2)$ per entity; a system that admits this in its limitations by reporting that a long-horizon benchmark would take on the order of a day of wall-clock time is reporting exactly this scaling [80]. Excitation-gated rewriting, in which a cheap rule decides whether a session contains anything new about an entity before the expensive rewrite is scheduled, is the write-path analogue of fan-out control and the most direct lever on this cost. Query-time construction [57] and LLM-free construction [28] attack the same cost from the other side by not building narratives at all.

7 Training signals and evaluation

7.1 Per-hop evidence as trajectory supervision

Neural cellular automata are trained on trajectories: a pool of intermediate states is kept, and the loss is applied at sampled steps so that the rule learns to reach and *hold* the target rather than pass through it [76]. Memory benchmarks already contain the equivalent supervision and do not use it. HotpotQA and MuSiQue annotate supporting facts per question [49, 75]; MemHop annotates, for every question, the chain of entities and the evidence sentence for each hop, derived programmatically from the scenario’s social graph rather than generated by an LLM [80]. Under the formalism, a hop annotation is a statement that cell e_k should be active by step k and that its activation should have been caused by e_{k-1} . That is a label for f at the level of individual (query, cell, neighbour-state) triples, which is exactly the level at which a decision model or a GNN rule is trained. Negative examples are the cells that are reachable but not on the path, which is the set a threshold automaton wrongly admits.

Three properties of this signal deserve notice. It is dense: a thousand five-hop questions yield several thousand positive triples and an order of magnitude more negatives. It does not require the LLM in the loop: the readout LLM is frozen, and the rule is fitted by supervised learning or by reinforcement learning against a proper scoring rule [36], with a temperature refit on held-out scenarios [43]. And it is transferable only with care: a rule fitted on synthetic social-network scenarios must be tested on held-out scenarios and on a benchmark with a different generating process, because the co-mention graph of a synthetic scenario has a structure that a real conversation need not share.

The multi-hop dense retriever is the existing precedent for training a cheap hop-conditioned rule on supporting-fact labels over documents [64]; Memora’s policy is the precedent for reinforcement

²Measured with the released context-assembly rule (per-cell profile plus up to eight residuals, no context cap) on stores built with a different identifier model than the paper’s; see Section 8.1 for why the cell count differs. Full-context baselines also enjoy prompt caching, since the prefix is identical across queries, an advantage that no memory-system evaluation we know of accounts for.

learning of a retrieval policy over memory units [67]; neither trains a per-cell rule of an iterated expansion on hop-indexed labels.

7.2 The one-shot baseline

Any claim that iteration helps must be tested against the same rule applied once. The same-compute control is a single pass in which the rule scores every cell against the query with no neighbourhood and the readout takes the top-k; this is a reranker [74], and it is the cheapest system that has access to the same information about each cell. If the iterated automaton does not beat it on accuracy at equal or smaller readout size, the neighbourhood and the schedule are contributing nothing and the gains, if any, come from the rule alone. The control is the memory-system version of the “one big network” question raised against continuous multi-agent communication: whether local iteration adds anything over a single global evaluation [27]. It should be reported alongside the full-context baseline, which bounds what any selection can achieve on a given corpus.

7.3 Evaluation pitfalls specific to this setting

Accuracy by hop depth conflates retrieval and reasoning; an oracle condition that supplies gold evidence directly separates them, and the profile-memory results show that even the oracle dips at two hops for reasons attributable to the answer model rather than to retrieval [80]. LLM-as-judge accuracy favours verbose and well-formatted answers, which has motivated debiased pairwise protocols that judge only the content unique to each answer [12, 91]. Token-overlap F1 penalises correct verbose answers. Benchmarks that ask whether a graph helps at all report that it does for reasoning and summarisation tasks and not for fact retrieval [41], which suggests that the value of a non-trivial neighbourhood should be reported per task type. Finally, cost must be reported in tokens and wall-clock time rather than in LLM calls: a system that makes one call per entity per session and rewrites a growing narrative in each is not cheaper than one that makes two small calls per turn, and call counts cannot distinguish them.

8 Pilot: iterated threshold expansion versus one-shot selection

We ran the falsifier of Section 7.2 on the one public benchmark with cell-level labels. The pilot answers a narrow question: given the same store and the same selection budget, does ProGraph’s iterated Stage 2 expansion select better cells than a one-shot ranker? Code, data mirror, cached stores and per-question results are at github.com/GA777777777/context-ca.

8.1 Protocol

Store. For MemHop scenarios 1–3 (5 people, 20 sessions and 100 questions each; 5.7–6.6K tokens of dialogue per scenario) we built the ProGraph store with the released write-path prompts verbatim [80], substituting `gpt-5.6-luna` for the paper’s `gpt-4o-mini` and keeping the paper’s embedder (all-MiniLM-L6-v2). All sessions were ingested before any question was asked; the read paths therefore see one fixed store per scenario. The identifier prompt asks for “all important entities ... anything worth tracking”, and with this model it returned on average 12 entities per session, so the stores hold 141, 153 and 148 cells (people, organisations, places, events, pets, projects) with 1.4–3.1K residuals; profile text alone is 4–5× the length of the dialogue it summarises. Cell count is thus a property of the identifier model, not of the data.

Selection rule	mean cells	readout tokens	readout $\div$ dialogue	chain recall [95% CI]	answer acc. [95% CI]
Stage 1 only ($H = 0$, top-5)	5.0	4.7K	$0.7\times$	0.253 [0.203, 0.303]	0.683 [0.633, 0.733]
Stage 2 as released, $\tau = 0.2$	140.8	56.9K	$9.1\times$	1.000	0.897 [0.860, 0.930]
Stage 2 as released, $\tau = 0.4$	108.1	48.4K	$7.7\times$	0.923 [0.890, 0.950]	—
Stage 2 per-candidate, $\tau = 0.2$	84.6	42.0K	$6.6\times$	0.837 [0.793, 0.877]	0.883 [0.847, 0.917]
Stage 2 per-candidate, $\tau = 0.3$	41.6	23.9K	$3.8\times$	0.503 [0.447, 0.560]	—
Stage 2 per-candidate, $\tau = 0.4$	16.3	10.8K	$1.7\times$	0.310 [0.257, 0.363]	—
One-shot cosine, $k = 20$	20.0	12.6K	$2.0\times$	0.323 [0.270, 0.373]	0.877 [0.840, 0.913]
One-shot cross-encoder, $k = 20$	20.0	15.1K	$2.4\times$	0.533 [0.473, 0.587]	0.890 [0.857, 0.923]
One-shot cross-encoder, $k = 40$	40.0	26.7K	$4.3\times$	0.800 [0.753, 0.843]	—
One-shot cosine, same budget as per-candidate	84.6	41.9K	$6.6\times$	0.820 [0.777, 0.863]	0.920 [0.887, 0.947]
One-shot cross-encoder, same budget	84.6	46.7K	$7.4\times$	0.987 [0.973, 0.997]	0.903 [0.870, 0.933]

Table 3: Budget, cell-level recall and answer accuracy on MemHop scenarios 1–3 ($n = 300$).

Comparison	chain recall	answer accuracy
cross-encoder one-shot – per-candidate iterated, same budget	+0.150 [+0.110, +0.190]	+0.020 [−0.007, +0.050]
cosine one-shot – per-candidate iterated, same budget	−0.017 [−0.033, −0.003]	+0.037 [+0.007, +0.067]
cross-encoder $k = 40$ – per-candidate iterated $\tau = 0.3$ (42 cells)	+0.297 [+0.233, +0.357]	—
cross-encoder $k = 20$ – Stage 2 as released (141 cells)	−0.467 [−0.527, −0.413]	−0.007 [−0.043, +0.033]

Table 4: Paired differences (positive favours the first rule).

Read paths. All variants share the store and ProGraph’s readout (profile plus up to eight query-ranked residuals per selected cell). *Iterated:* ProGraph Stage 1 (top-5 by cosine with name boost) followed by Stage 2 as released (gate τ on the best candidate, then admit all co-mentioned candidates; $H = 5$), for $\tau \in \{0.2, 0.3, 0.4\}$; and the per-candidate variant the paper describes (each candidate must clear τ itself). *One-shot:* every cell scored once against the query by the same cosine scorer, or by a cross-encoder reranker (`ms-marco-MiniLM-L-6-v2`), taking the top- k . The same-budget control sets k , per question, to the number of cells the per-candidate iterated variant selected. Nothing is trained; there is no held-out split because there is nothing to overfit.

Labels and metrics. Each MemHop question’s decomposition names one evidence observation per hop, and the observation names its person; *chain recall* is 1 when every hop’s person is covered by a selected cell (full name or first token, since the identifier registers both “Alice” and “Alice Chen”). We also record the selected-set size, the readout in tokens, the per-step trace, and answer accuracy under ProGraph’s answer prompt and its *strict* judge prompt with the same LLM; the ProGraph paper’s headline numbers use the more permissive LightMem judge, so accuracies here are not comparable to its reported 80.1%. Confidence intervals are 95% percentile bootstraps over the 300 questions; paired differences are bootstrapped over per-question differences.

8.2 Results

Answer accuracy was measured for seven rules (the two Stage 2 rules at $\tau = 0.2$, Stage 1 only, the two $k = 20$ rankers, and the two same-budget rankers); rows marked — were not answered.

The released rule is a percolating automaton. At $\tau = 0.2$ the active set grows from 5 cells to 103 after one step and to 141 after two, then stops: 96% of the store. Raising τ to 0.4 removes 33 cells and costs 8 points of recall. The per-candidate rule the paper describes admits 85 cells at $\tau = 0.2$ and collapses to 42 at $\tau = 0.3$ and 16 at $\tau = 0.4$, with recall falling from 0.84 to 0.50 to 0.31, the last barely above Stage 1 alone. No threshold lands in the region of small budget and high recall; the process either admits most of the store or almost nothing beyond the seeds. The readout at $\tau = 0.2$ is 9.1 times the dialogue it was built from.

Iteration does not beat a one-shot ranker at matched budget. With the same scorer (cosine) the iterated rule and the one-shot ranker are indistinguishable on recall (difference -0.017) and the one-shot is slightly better on accuracy. With a scorer that reads the cell text (a 22M-parameter cross-encoder) the one-shot ranker leads the iterated rule by 15 points of recall at the same budget, and by 30 points against the $\tau = 0.3$ iterated rule at a comparable budget. The neighbourhood structure contributed nothing that a better per-cell score did not.

Answer accuracy saturates at about 20 well-ranked cells. Every rule that selects 20 or more cells scores between 0.877 and 0.920, with overlapping intervals; Stage 1 alone scores 0.683. The full-store readout of the released rule is not measurably better than the cross-encoder’s top 20 (-0.007 [-0.043 , $+0.033$]) at 3.8 times the tokens.

Cell-level chain recall is a weak proxy. Cosine top-20 covers the full gold chain on 32% of questions yet answers 88% correctly. ProGraph profiles are narratives that carry cross-entity facts, so one profile frequently contains the whole chain; entity-level coverage undercounts information coverage. This is the mechanism the ProGraph paper credits for its multi-hop gains, and it is also why a one-shot ranker over such profiles suffices.

8.3 Cost and caveats

The three stores cost 1.2K write-path calls, 1.5M input and 1.0M output tokens. Answering 300 questions under eleven rules cost 2.1K calls and 65M input tokens, almost all of it readout: the released rule’s 57K-token contexts hit the provider’s per-minute token limit and had to be paced. At the provider’s list price the pilot cost about \$15, of which the write path was under \$2. Answer and judge calls sent no reasoning-effort parameter (the model’s default) for the first pass and **none** for the 174 calls retried after rate limiting; completion lengths averaged about 70 tokens in both populations, consistent with no hidden reasoning in either.

Caveats. The scenarios are synthetic, five-person social networks; the co-mention graph of real dialogue may be sparser. One identifier model was used, and the cell count it produced is an order of magnitude above what the ProGraph paper implies for **gpt-4o-mini**; the percolation behaviour depends on the density of the co-mention graph and should be re-measured at a smaller cell count. The answer model is strong enough that retrieval differences vanish above 20 cells; a weaker answer model, or questions whose evidence is not duplicated across profiles, could re-open the gap. Accuracy is a single LLM judge with the ProGraph prompt. Nothing here tests a learned rule; the pilot only establishes that the hand-set threshold rule is dominated by a one-shot reader, which is the precondition for the agenda in Section 10.

8.4 Where could the neighbourhood still matter? Three follow-ups

The pilot rejects a narrow claim: iteration over a string co-mention neighbourhood with a hand-set gate, on cells that already contain their neighbours’ facts, read by a global LLM. Each clause is a place the neighbourhood might still earn its keep. We tested three of them with the same stores and no further LLM calls, and a fourth (Section 8.5) with a small number of calls. Paired differences are bootstrapped over the same 300 questions.

Cells that do not pre-materialise neighbours. ProGraph’s profile is a narrative that names the neighbours; a one-shot reader gets the chain for free. We removed the narrative and used only the residual bundle of each entity as the cell text (embedding = mean of its residual embeddings). At matched budget (95 cells) the iterated rule and the one-shot cosine ranker are identical (difference 0.000) and the cross-encoder is two points *behind* (-0.020 [-0.037 , -0.007]); the narrative-free cosine ranker is, unexpectedly, far *better* than on narratives ($+0.407$ [$+0.353$, $+0.460$])

at $k = 40$, reaching 0.99 chain recall at $k = 60$). We then went to the atomic extreme, one residual per cell as in Mem0. A one-shot cosine ranker over atoms reaches 0.937 chain recall with 80 atoms (1.8K tokens); a one-hop expansion from 20 atoms through the entities they name reaches 76 atoms and 0.777, sixteen points *worse* than one-shot at a comparable count (+0.160 [+0.117, +0.207] for one-shot). The neighbourhood does not help even when no narrative exists, because ProGraph’s residual prompt asks for self-contained facts naming the entity and lists cross-person facts as a category: the neighbour is written into the atom at extraction time. Any write path that stores self-contained propositions has this property.

The atomic result looked like a $25\times$ token saving until we measured answer accuracy. With the top 80 or 160 residuals as the *only* context (no profiles), accuracy is 0.523 [0.467, 0.580] and 0.617 [0.557, 0.670], against 0.890 for twenty profiles; the loss is entirely on multi-hop questions ($K \geq 2$: 0.31–0.61 versus 0.92 at $K = 1$). Entity coverage is not fact coverage: the atoms cover the right people but the sentence that links them lives in the narrative. This is the strongest evidence in our data for the two-layer decomposition of Section 5.2, and the clearest demonstration that cell-level chain recall is a weak proxy in both directions.

Distractors. We merged the three stores into one of 442 cells and asked each scenario’s questions against the union; a cell from another scenario can never satisfy a hop. The released rule now selects 423 cells, two thirds of them from the wrong scenario: co-mention leaks across scenarios through shared first names and through noise entities (“10k”, “animal”, “boston”) that the identifier registered as cells. The per-candidate rule needs 193 cells to hold the recall it had at 85 (difference 0.000; $2.3\times$ the cells). The cross-encoder one-shot ranker is nearly immune (5% distractors at $k = 40$, recall -0.093 [-0.130 , -0.060] versus the single store), the cosine ranker is not (29%). At matched budget the one-shot cross-encoder leads by +0.130 [+0.093, +0.170]. A string co-mention neighbourhood is therefore not a filter but an amplifier of distractors; a neighbourhood earns its keep against distractors only if its edges are cleaner than the ranker’s scores, which typed relations might be and co-mention is not.

Edge-conditioned gating. The rules above score a candidate against the query alone. The neighbourhood’s distinctive input is the *pair*: the text in the seed’s cell that mentions the candidate. Section 8.6 tests whether a rule that reads this pair beats a rule that reads the candidate alone.

8.5 The write path: propagation under a bounded window

The read path has a global reader, so locality can only cost. The write path has none: when a session arrives, no component reads the whole store, and an update reaches a cell only if some rule carries it there. This is where a cellular automaton is the natural object, and we tested it directly.

Protocol. Into each scenario’s completed store we injected a 21st session in which person A tells a third person that A and B are no longer in their stored relationship (roommates, lab partners, co-founders, ...) and that B has moved to another city; four relationships per scenario, twelve injections. Before injection an LLM judge marked every cell that mentions A or B as *affected* if its text asserted the relationship (89 cells in total, 1–28 per injection; the third parties are pets, shared friends, events and places whose profiles record the pair). We then ran five write rules from the same pre-injection store and judged, per affected cell, whether the profile still asserted the old fact, whether it reflected the change, and whether any residual still asserted the old fact:

- **R0**, ProGraph as released: rewrite the entities the identifier finds in the session (A, B, the interlocutor, and whatever else is named).
- **R1**, co-mention: R0 plus every cell whose text mentions both A and B.
- **R2**, excitation with a gate: spread from A and B over the co-mention neighbourhood for two hops, then rewrite the excited cells a cross-encoder scores as related to the change sentence.

Rule	rewrites / injection	tokens / injection (in / out)	affected cells never rewritten	profile still stale	profile has new fact	residual still stale	collateral rewrites (total)
R0 as released	8.4	14.9K / 9.8K	54 / 89	51 / 89	33 / 89	62 / 89	66
R1 co-mention	41.1	56.7K / 34.6K	0 / 89	2 / 89	87 / 89	39 / 89	404
R2 excitation + gate	38.0	47.4K / 26.7K	5 / 89	6 / 89	83 / 89	39 / 89	372
R3 oracle	12.9	21.5K / 14.2K	0 / 89	1 / 89	87 / 89	38 / 89	66
R4 oracle + eviction	12.9	39.3K / 14.4K	0 / 89	1 / 89	88 / 89	15 / 89	66

Table 5: Propagation of a state change (12 injections, 89 affected cells).

- **R3**, oracle: rewrite exactly the affected cells (the target).
- **R4**, oracle plus eviction: R3, then one LLM call per affected cell that lists which of its residuals assert the outdated fact; those are deleted (the contradiction transition of Section 5.4).

The released rule leaves the store inconsistent. Sixty-one percent of the cells that held the old relationship were never touched, because they were not named in the session; per-injection profile consistency is 0.54. The update reached exactly the cells inside the session’s window and no others. In the worst case (a relationship recorded in 28 cells) 21 were missed.

Propagation restores consistency, and the neighbourhood is what makes it possible. Every rule that carries the update along edges brings consistency above 0.95. Unlike the read path, there is no one-shot alternative here: nothing else visits the cells that hold the stale fact. The neighbourhood’s contribution is reach.

Selectivity is the open problem, and it is the same problem as on the read path. Two-hop excitation touches 139–151 cells, essentially the store; the cross-encoder gate admits 38 of them per injection and the co-mention rule 41, against an oracle of 12.9 (7.4 affected plus 5.5 that the session names). The gate, hand-set and untrained, removes almost none of the collateral. The gap between 38 and 13 rewrites is a classification problem over (change sentence, cell text) pairs, and the experiment produced its own training set: about 1,700 excited cells with judged labels.

Residuals are a second consistency leak, and a local contradiction rule closes most of it. Even the oracle rewrite leaves 43% of affected cells with a residual asserting the old fact, because the write path only appends. One eviction call per affected cell brings this to 17%, at about 18K extra input tokens per injection. This is the one intervention in this section that improves consistency without touching more cells.

Caveats: twelve synthetic injections in five-person scenarios; a single LLM as writer and judge; relationships that end cleanly, whereas real updates are often partial. The cost of the section was under three dollars.

8.6 The pair is the signal: edge-conditioned gating at small budgets

Every rule in Sections 8.2–8.4 scores a cell against the query in isolation; the neighbourhood only decides *which* cells get scored. If that is all a neighbourhood does, a global ranker that scores every cell must dominate it, and it did. But a neighbourhood carries one input a global ranker cannot construct: for each candidate, the passage in the seed’s cell that mentions it. We tested whether a rule that reads that passage is a different rule.

Protocol. From the same five Stage 1 seeds, a budgeted expansion adds up to five candidates per step along co-mention edges until k cells are selected ($k \in \{10, 20, 40\}$), ranking the frontier by one of three gates: cosine of the candidate profile to the query; the cross-encoder on (query, candidate profile); and the cross-encoder on (query, seed-window $\parallel$ candidate profile), where the seed window is the 220 characters of the seed’s profile around the first mention of the candidate’s name. The control is the one-shot cross-encoder over all cells at the same k . Same 22M-parameter cross-encoder throughout, untrained for the task; same stores as Section 8.2; chain recall over the

Rule	k = 10	k = 20	k = 40	k = 40, K = 4	k = 40, K = 5
One-shot cross-encoder (control)	0.367	0.523	0.733	0.52	0.53
Expansion, cosine gate	0.273	0.333	0.447	0.11	0.11
Expansion, cross-encoder on candidate alone	0.383	0.520	0.743	0.52	0.58
Expansion, cross-encoder on seed-window candidate	0.430	0.653	0.897	0.85	0.61
Pair gate – one-shot, all K	+0.063 [+0.017, +0.107]	+0.130 [+0.080, +0.180]	+0.163 [+0.110, +0.220]		
Pair gate – one-shot, $K \geq 2$	+0.044 [-0.004, +0.098]	+0.173 [+0.107, +0.240]	+0.218 [+0.147, +0.289]		
Candidate-only gate – one-shot, all K	+0.017 [-0.013, +0.050]	-0.003 [-0.020, +0.013]	+0.010 [+0.000, +0.020]		

Table 6: Budgeted expansion with three gates versus one-shot ranking at matched k ($n = 300$; $K \geq 2$ subset $n = 225$).

300 questions, paired bootstrap for differences.

The neighbourhood as a candidate generator adds nothing. Expansion gated by the cross-encoder on the candidate alone is indistinguishable from the one-shot ranker at every budget (all three intervals contain zero), reproducing the pilot’s conclusion with a different search procedure. Expansion gated by cosine is far worse, as in Section 8.2.

The neighbourhood as a source of pair text changes the rule. The same encoder, the same edges and the same budget, with the seed window prepended to the candidate, gains 13 points of chain recall at $k = 20$ and 16 at $k = 40$ over the one-shot ranker, and 15 over the candidate-only gate. The gain grows with hop depth: at $k = 40$ four-hop recall rises from 0.52 to 0.85. On the multi-hop subset the pair gate leads the one-shot ranker by 0.218 [0.147, 0.289] at $k = 40$. At $k = 10$ the gain is present but small, because five of the ten slots are the seeds.

Why a global ranker cannot do this. The pair gate’s input is a function of an edge, not of a cell: it exists only because the expansion knows which seed reached which candidate and can look up the sentence that connects them. A one-shot ranker over cells has no such sentence to read; a one-shot ranker over all (cell, cell) pairs would score $|V|^2$ inputs, which at 147 cells is 21,000 cross-encoder calls per query against roughly 150 for the expansion. The neighbourhood’s contribution is therefore not reach (Section 8.2 showed reach is cheap to get and dangerous to have) but *the narrowing of the pair space to edges that exist*, which makes reading relation text affordable. This is the first positive result for the neighbourhood on the read path, and it is exactly the input that the typed decision models of Section 4.5 accept: the neighbour’s message enters the rule as text alongside the candidate, within a bounded window.

What this does and does not establish. It establishes that a local rule with access to edge text beats the best global rule without it, at matched budget, with an untrained encoder, on co-mention edges that Section 8.2 showed to be noisy. It does not establish that the gain survives real dialogue, where profiles may not name the bridge; nor that answer accuracy follows, since Section 8.2 found accuracy saturating near twenty well-chosen cells; nor what a trained pair gate would reach. It sets the bar for the latter: 0.897 at forty cells, from a reranker that has never seen a MemHop question. A trained pair gate, answer-level evaluation of the selected cells, and a second benchmark are in progress and will appear in a later version of this paper.

9 Relation to latent multi-agent communication

A Context-CA in which every cell is itself a language model is a multi-agent system on a graph, and the transition rule becomes inter-agent communication. That literature has recently moved from text channels to latent ones: agents exchange hidden states or KV-cache entries instead of tokens [55], a unifying framework covers eighteen such methods [13], convergent merging of caches has been proposed for order-independent aggregation [23], and a causal audit finds that the mechanism by which latent channels help changes with model scale [15]. Benchmarks of LLM collectives on

classical distributed-coordination problems show that text-channel coordination degrades sharply with the number of agents [5], and pruning studies find that most inter-agent text is dispensable [4]. The lineage runs back to differentiable inter-agent channels in multi-agent reinforcement learning [22, 25] and, for the single-model case, to continuous latent reasoning [18].

The Context-CA of this survey sits at the opposite end of the same spectrum. Its cells are memory units, not agents; its channel is a scalar activation or a one-line message, not a hidden state; and its rule is a small encoder, not an LLM. Three consequences follow. The channel is narrow, so a cell cannot tell *why* a neighbour is active, only *that* it is, and deep chains may need a one-line textual message as a visible commitment, which is the hybrid protocol the latent-communication literature also arrives at for auditability [13]. Stability is cheap to measure, because the state is a vector of scalars whose drift across steps can be plotted directly, whereas hidden-state drift across hops is an open problem in the latent setting [15]. And the readout language model can be an API model, because nothing touches its hidden states; latent communication is confined to open-weight models by construction. The empirical case for discrete over continuous messages between heterogeneous agents [27] cuts the other way here: a calibrated scalar is the *most* discrete useful channel, and the question is whether it is too narrow rather than too wide.

10 Open problems and an experimental agenda

We list the questions the formalism makes askable and that the literature does not answer.

1. **Does an iterated rule beat a one-shot reranker?** The central falsifier (Section 7.2). Section 8 answers it in two parts. A rule that scores cells in isolation does not: at matched budget a one-shot cross-encoder selects better cells and iteration adds nothing over its own scorer, under narratives, residual bundles, atoms and distractors. A rule that scores the *edge* does: the same encoder reading the seed’s sentence about the candidate gains 16 points at forty cells and 22 on multi-hop questions (Section 8.6). What remains open is whether the gain survives real dialogue and a trained gate, and whether answer accuracy follows.
2. **Can a typed decision model serve as the rule?** Zero-shot it is weaker than a cosine threshold on the vendor’s own numbers; the question is whether a fine-tune on hop labels closes the gap and whether it transfers across benchmarks. The 512–1024-token window is a constraint on cell design, not on the method.
3. **Fan-out control as a local rule.** Lateral inhibition is the only fan-out mechanism expressible inside *f*. Its effect on the active-set size versus hop depth, and whether it can be learned rather than hand-set, is untested outside one system [87].
4. **Stability of continuous activation on memory graphs.** Attractor analyses of trained NCAs exist only on grids and find oscillation [77, 78]. A memory graph with a damped residual update needs its own analysis, or a monotone rule with a step budget as the safe default.
5. **Excitation-gated writing.** Section 8.5 shows that propagation along the neighbourhood is necessary for consistency and that hand-set gates admit three times the oracle’s rewrites. The open question is whether a small model trained on (change, cell) pairs, of which that experiment produced about 1,700 with judged labels, can approach the oracle’s 13 rewrites per update without missing affected cells.
6. **Contradiction as a local transition.** A one-call-per-cell eviction rule cut stale residuals from 43% to 17% (Section 8.5); what closes the remaining gap, and whether a non-generative verifier can replace the LLM call, is untested.
7. **Benchmarks with cell-level labels beyond synthetic scenarios.** MemHop’s per-hop annotations are derived from a synthetic graph; the field lacks a conversational multi-hop

benchmark whose hop labels come from real dialogue.

8. **Heterogeneous neighbourhoods.** Co-mention of names is the weakest usable neighbourhood. Typed relations from a knowledge graph, cross-modal links between documents and code, and hierarchical containment give the rule an edge type to condition on, at the cost of a write-time extraction step whose errors the rule must tolerate.
9. **Cost reporting.** Tokens, wall-clock time, and prompt-cache eligibility, per read and per write, on corpora that span at least two orders of magnitude in size, so that the crossover point at which selection beats full context is measured rather than assumed.

A minimal experimental sequence that addresses items 1–4 is: reproduce a threshold-expansion read path and log per-step active sets against gold hops (done, Section 8); replace the threshold with a zero-shot decision model and confirm the expected degradation; fine-tune on hop labels from disjoint scenarios and test on the held-out ones and on a second benchmark; add a damped residual update with lateral inhibition and compare, at matched compute, against the one-shot reranker and the full-context baseline. The pilot fixes the bar the learned rule has to clear: 0.987 chain recall at 85 cells from a one-shot cross-encoder, and 0.897 at 40 from an untrained pair gate (Section 8.6). Each stage has a pre-registered failure condition, and the sequence can be run with a frozen API model as the readout.

11 Conclusion

The memory systems built for LLM agents over the last two years share more structure than their descriptions suggest. Read as cellular automata on a graph of memory units, they differ in the choice of cell, neighbourhood, state, rule, schedule and readout, and the choices they have in common are as informative as the ones they vary: binary state, a rule that is either a threshold or a generation, fan-out bounded by a step budget rather than by the rule, and no training signal despite labelled trajectories sitting in the benchmarks. The formalism does not by itself produce a better system. It produces a list of experiments, and the ones we ran divide the question cleanly. On the read path, where a language model reads everything selected, a neighbourhood that only decides which cells to score is dominated by scoring every cell; the same neighbourhood becomes an advantage the moment the rule reads the text of the edge, because that input exists only along edges and is unaffordable without them. On the write path, where nothing reads everything, the neighbourhood is the only route by which an update reaches the cells that need it, and the open problem is selectivity. Both halves point at the same missing component: a small rule that reads a bounded window containing a cell and its neighbour’s message about it, trained on the per-hop and per-update labels the experiments themselves produce. The candidates for that rule exist; their fitness for the role is, as of this writing, the next experiment.

References

- [1] Dadhich, G. (2026). Agentic Context Management: Solving Agent Memory and Cost by Treating Them as Lifecycle and Architecture Problems. arXiv:2607.21503.
- [2] Kang, Chen, Han, et al. (2026). ACON: Optimizing Context Compression for Long-horizon LLM Agents. ICML 2026. arXiv:2510.00615.
- [3] Anderson, J. R., Bothell, Byrne, et al. (2004). An Integrated Theory of the Mind. *Psychological Review* 111(4), 1036–1060.

- [4] Zhang, Yue, Li, et al. (2025). Cut the Crap: An Economical Communication Pipeline for LLM-based Multi-Agent Systems. ICLR 2025. arXiv:2410.02506.
- [5] Grötschla, Müller, Tönshoff, et al. (2025). AgentsNet: Coordination and Collaborative Reasoning in Multi-Agent LLMs. arXiv:2507.08616.
- [6] Zha, Yang, Li, et al. (2023). AlignScore: Evaluating Factual Consistency with a Unified Alignment Function. ACL 2023. arXiv:2305.16739.
- [7] Zhang, G., Jiang, Wang, et al. (2026). Adaptive Memory Admission Control for LLM Agents. ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems. arXiv:2603.04549.
- [8] Xu, Liang, Mei, et al. (2025). A-MEM: Agentic Memory for LLM Agents. NeurIPS 2025. arXiv:2502.12110.
- [9] Anderson, J. R. (1983). A spreading activation theory of memory. *Journal of Verbal Learning and Verbal Behavior* 22(3), 261–295.
- [10] Zhu, W., Chen, Wang, et al. (2026). AriadneMem: Threading the Maze of Lifelong Memory for LLM Agents. arXiv:2603.03290.
- [11] Hou, Yuan, Zhou, et al. (2026). AtomicRAG: Atom-Entity Graphs for Retrieval-Augmented Generation. arXiv:2604.20844.
- [12] Microsoft Research (2025–2026). BenchmarkQED: Automated benchmarking of RAG systems (software), including the unbiased pairwise scoring command (July 2026). github.com/microsoft/benchmark-qed.
- [13] Liu, Y. (2026). Beyond tokens: a unified framework for latent communication in LLM-based multi-agent systems. arXiv:2606.05711.
- [14] Brin, S., Page, L. (1998). The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems* 30(1–7), 107–117.
- [15] Zhang, H., Emu, M. (2026). Do Latent Channels Actually Communicate? A Causal Audit of Latent Multi-Agent LLM. arXiv:2607.26773.
- [16] Chalupa, Leath, Reich (1979). Bootstrap percolation on a Bethe lattice. *Journal of Physics C* 12(1), L31.
- [17] Sumers, Yao, Narasimhan, et al. (2024). Cognitive Architectures for Language Agents. TMLR 2024. arXiv:2309.02427.
- [18] Hao, Sukhbaatar, Su, et al. (2025). Training Large Language Models to Reason in a Continuous Latent Space. COLM 2025. arXiv:2412.06769.
- [19] Khattab, O., Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. SIGIR 2020. arXiv:2004.12832.
- [20] Li, Hou, Jing, et al. (2026). CompactionRL: Reinforcement Learning with Context Compaction for Long-Horizon Agents. arXiv:2607.05378.
- [21] Collins, A. M., Loftus, E. F. (1975). A spreading-activation theory of semantic processing. *Psychological Review* 82(6), 407–428.

- [22] Sukhbaatar, Szlam, Fergus (2016). Learning Multiagent Communication with Backpropagation. NIPS 2016. arXiv:1605.07736.
- [23] Baquero, C., Brito, L. (2026). Cache Merging as a Convergent Replicated State for Multi-Agent Latent Reasoning. arXiv:2607.01308.
- [24] Barandiaran, Stovold (2025). Growing Reservoirs with Developmental Graph Cellular Automata. ALIFE 2025. arXiv:2508.08091.
- [25] Foerster, Assael, de Freitas, et al. (2016). Learning to Communicate with Deep Multi-Agent Reinforcement Learning. NIPS 2016. arXiv:1605.06676.
- [26] Dijkstra, E. W. (1974). Self-stabilizing systems in spite of distributed control. Communications of the ACM 17(11), 643–644.
- [27] Chen, H., Jang, Zhou, et al. (2023). Discrete Messages Improve Communication Efficiency among Isolated Intelligent Agents. arXiv:2312.15985.
- [28] Cui, Cao, Wen, et al. (2026). EdgeMem: LLM-Free Agent Memory Construction and Retrieval via Evidence-Preserving Multi-Anchor Hypergraph. arXiv:2609.05553.
- [29] Gala, Grattarola, Quaeghebeur (2024). E(n)-equivariant Graph Neural Cellular Automata. TMLR 2024. arXiv:2301.10497.
- [30] Guichard, Reimers, Kvalsund, et al. (2025). EngramNCA: a Neural Cellular Automaton Model of Memory Transfer. ALIFE 2025. arXiv:2504.11855.
- [31] Hu, C., Gao, Zhou, et al. (2026). EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning. arXiv:2601.02163.
- [32] Wei, Peng, Dong, et al. (2026). FadeMem: Biologically-Inspired Forgetting for Efficient Agent Memory. arXiv:2601.18642.
- [33] Chen, L., Zaharia, Zou (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176.
- [34] Park, O’Brien, Cai, et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023. arXiv:2304.03442.
- [35] Gilmer, Schoenholz, Riley, et al. (2017). Neural Message Passing for Quantum Chemistry. ICML 2017, PMLR 70, 1263–1272.
- [36] Gneiting, T., Raftery, A. E. (2007). Strictly Proper Scoring Rules, Prediction, and Estimation. JASA 102(477), 359–378.
- [37] Grattarola, Livi, Alippi (2021). Learning Graph Cellular Automata. NeurIPS 2021. arXiv:2110.14237.
- [38] Mavromatis, C., Karypis, G. (2025). GNN-RAG: Graph Neural Retrieval for Efficient Large Language Model Reasoning on Knowledge Graphs. Findings of ACL 2025, 16682–16699. arXiv:2405.20139.
- [39] Edge, Trinh, Cheng, et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130.

- [40] Microsoft Research (2024). GraphRAG: Improving global search via dynamic community selection. Blog, 15 Nov 2024.
- [41] Xiang, Wu, Zhang, et al. (2026). When to use Graphs in RAG: A Comprehensive Analysis for Graph Retrieval-Augmented Generation. ICLR 2026. arXiv:2506.05690.
- [42] He, Tian, Sun, et al. (2024). G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering. NeurIPS 2024. arXiv:2402.07630.
- [43] Guo, Pleiss, Sun, et al. (2017). On Calibration of Modern Neural Networks. ICML 2017. arXiv:1706.04599.
- [44] Zhang, Z., Sheng, Zhou, et al. (2023). H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. NeurIPS 2023. arXiv:2306.14048.
- [45] Jiménez Gutiérrez, Shu, Gu, et al. (2024). HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. NeurIPS 2024. arXiv:2405.14831.
- [46] Jiménez Gutiérrez, Shu, Qi, et al. (2025). From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. ICML 2025. arXiv:2502.14802.
- [47] Honda, Fujita, Zempo, et al. (2025). Human-Like Remembering and Forgetting in LLM Agents: An ACT-R-Inspired Memory Architecture. HAI 2025, 229–237.
- [48] Ramsauer, Schäfl, Lehner, et al. (2021). Hopfield Networks is All You Need. ICLR 2021. arXiv:2008.02217.
- [49] Yang, Qi, Zhang, et al. (2018). HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. EMNLP 2018.
- [50] Trivedi, Balasubramanian, Khot, et al. (2023). Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. ACL 2023. arXiv:2212.10509.
- [51] Jeh, G., Widom, J. (2003). Scaling personalized web search. WWW 2003.
- [52] TypeSafe AI (2026). Introducing System One Models & Jev. Blog, 15 Sep 2026. Commercial API; architecture undisclosed.
- [53] Kempe, Kleinberg, Tardos (2003). Maximizing the spread of influence through a social network. KDD 2003, 137–146.
- [54] Huang, Zhang, Xiao (2025). KET-RAG: A Cost-Efficient Multi-Granular Indexing Framework for Graph-RAG. KDD 2025. arXiv:2502.09304.
- [55] Zou, Qiu, Li, et al. (2025). Latent Collaboration in Multi-Agent Systems. ICML 2026 (Spotlight). arXiv:2511.20639.
- [56] Convai Innovations (2026). Laya: non-autoregressive typed decision models (software, Apache-2.0), released 18 Sep 2026. github.com/NandhaKishorM/laya; huggingface.co/convaiinnovations/laya. No peer-reviewed description.
- [57] Yu, J., Zhao, Zhang, et al. (2026). LazyMem: Retrieve Broadly, Construct Selectively for Efficient Long-Term Agent Memory. arXiv:2607.22690.

- [58] Zhang, J., Zhang, C., Chen, et al. (2026). Lightweight LLM Agent Memory with Small Language Models. ACL 2026. arXiv:2604.07798.
- [59] Guo, Xia, Yu, et al. (2024). LightRAG: Simple and Fast Retrieval-Augmented Generation. arXiv:2410.05779.
- [60] Jiang, H., Wu, Lin, et al. (2023). LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. EMNLP 2023. arXiv:2310.05736.
- [61] Maharana, Lee, Tulyakov, et al. (2024). Evaluating Very Long-Term Conversational Memory of LLM Agents. ACL 2024, 13851–13870. arXiv:2402.17753.
- [62] Wu, D., Wang, Yu, et al. (2025). LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. ICLR 2025. arXiv:2410.10813.
- [63] Liu, N. F., Lin, Hewitt, et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. TACL 12, 157–173. arXiv:2307.03172.
- [64] Xiong, Li, Iyer, et al. (2021). Answering Complex Open-Domain Questions with Multi-Hop Dense Retrieval. ICLR 2021. arXiv:2009.12756.
- [65] Chhikara, Khant, Aryan, et al. (2025). Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413.
- [66] Zhang, J., Chen, Ma, et al. (2026). Beyond Similarity: Trustworthy Memory Search for Personal AI Agents. arXiv:2606.06054.
- [67] Xia, Zhang, Dixit, et al. (2026). Memora: A Harmonic Memory Representation Balancing Abstraction and Specificity. ICML 2026. arXiv:2602.03315.
- [68] Zhong, Guo, Gao, et al. (2024). MemoryBank: Enhancing Large Language Models with Long-Term Memory. AAAI 2024, 19724–19731. arXiv:2305.10250.
- [69] Hu, Y., Wang, Y., McAuley, J. (2025). Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. arXiv:2507.05257.
- [70] Tang, Laban, Durrett (2024). MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents. EMNLP 2024. arXiv:2404.10774.
- [71] Marone, Weller, Fleshman, et al. (2025). mmBERT: A Modern Multilingual Encoder with Annealed Language Learning. arXiv:2509.06888.
- [72] Tang, Z., Yu, Xiao, et al. (2026). Mnemis: Dual-Route Retrieval on Hierarchical Graphs for Long-Term LLM Memory. ACL 2026. arXiv:2602.15313.
- [73] Warner, Chaffin, Clavié, et al. (2024). Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference. arXiv:2412.13663.
- [74] Nogueira, R., Cho, K. (2019). Passage Re-ranking with BERT. arXiv:1901.04085.
- [75] Trivedi, Balasubramanian, Khot, et al. (2022). MuSiQue: Multihop Questions via Single-hop Question Composition. TACL 10, 539–554.

- [76] Mordvintsev, Randazzo, Niklasson, et al. (2020). Growing Neural Cellular Automata. *Distill*.
- [77] Kvalsund, M.-K., Stovold, J. (2026). Stability and Geometry of Attractors in Neural Cellular Automata. *arXiv:2604.12720*.
- [78] Stovold, Kvalsund, Ludwig, et al. (2026). Visualising the Attractor Landscape of Neural Cellular Automata. *ALIFE 2026*. *arXiv:2604.10639*.
- [79] Pavlović, Krész, Hajdu (2025). Leveraging Spreading Activation for Improved Document Retrieval in Knowledge-Graph-Based RAG Systems. *arXiv:2512.15922*.
- [80] Zhu, S. (2026). Profile-Graph Memory for LLM Agents: Implicit Cross-Entity Traversal through Narrative Profiles. *arXiv:2607.19359*.
- [81] Sarthi, Abdullah, Tuli, et al. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. *ICLR 2024*. *arXiv:2401.18059*.
- [82] Ong, Almahairi, Wu, et al. (2025). RouteLLM: Learning to Route LLMs with Preference Data. *ICLR 2025*. *arXiv:2406.18665*.
- [83] Press, Zhang, Min, et al. (2023). Measuring and Narrowing the Compositionality Gap in Language Models. *Findings of EMNLP 2023*. *arXiv:2210.03350*.
- [84] Li, Y., Dong, Lin, et al. (2023). Compressing Context to Enhance Inference Efficiency of Large Language Models. *EMNLP 2023*. *arXiv:2310.06201*.
- [85] Li, Y., Huang, Yang, et al. (2024). SnapKV: LLM Knows What You are Looking for Before Generation. *NeurIPS 2024*. *arXiv:2404.14469*.
- [86] Xiao, Tian, Chen, et al. (2024). Efficient Streaming Language Models with Attention Sinks. *ICLR 2024*. *arXiv:2309.17453*.
- [87] Jiang, Chen, Pan, et al. (2026). SYNAPSE: Empowering LLM Agents with Episodic-Semantic Memory via Spreading Activation. *arXiv:2601.02744*.
- [88] Sun, J., Xu, Tang, et al. (2024). Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. *ICLR 2024*. *arXiv:2307.07697*.
- [89] Sun, J., Zhu, Yao, et al. (2026). Rethinking How to Remember: Beyond Atomic Facts in Lifelong LLM Agent Memory. *arXiv:2605.19952*.
- [90] Su, Guo, Hou, et al. (2026). Beyond Dialogue Time: Temporal Semantic Memory for Personalized LLM Agents. *arXiv:2601.07468*.
- [91] Zeng, Luo, Lin, et al. (2025). How Significant Are the Real Performance Gains? An Unbiased Evaluation Framework for GraphRAG. *arXiv:2506.06331*.
- [92] Viola, P., Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features. *CVPR 2001*.
- [93] Wolfram, S. (1984). Universality and complexity in cellular automata. *Physica D* 10(1–2), 1–35.
- [94] Mei, Yao, Ge, et al. (2025). A Survey of Context Engineering for Large Language Models. *arXiv:2507.13334*.

- [95] Packer, Wooders, Lin, et al. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.
- [96] Nguyen, Qiu, Chen, et al. (2026). Graph-Based Personalized Memory for LLM Agents: Representation, Evolution, Retrieval, and Evaluation. ICKG 2026. arXiv:2609.08599.
- [97] Hu, Y., Liu, S., Yue, Y., et al. (2025). Memory in the Age of AI Agents. arXiv:2512.13564.
- [98] Rasmussen, Paliychuk, Beauvais, et al. (2025). Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956.